Python projects with uv

Software Management & Development



IE University · MFT & MCSDI

From a script to a project

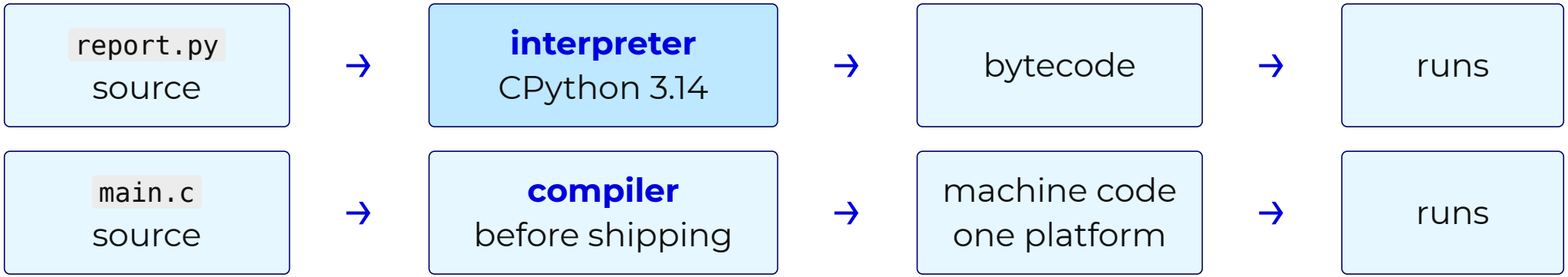
What a project needs, and where uv keeps it:

A declared **Python version**	→	`requires-python`, `.python-version`
Its **own** installed libraries	→	`.venv`
A **precise list** of them, with versions	→	`uv.lock`
A **standard way** to run it	→	`uv run ledger`

> Today `ledger` goes from `balance.sh` to a Python project that anyone can rebuild with one command.

The interpreter

Python is interpreted



- C, Go and Rust ship machine code
- Python ships **source**: the interpreter is part of the environment, so its **version** matters

Python versions

3 . **14** . **2**

major minor patch

- A new **minor** version every October, supported for 5 years
- **Patch** versions only fix bugs
- Old code runs on new Python. **Not the reverse**

New code on old Python

Code uses	Since	On an older Python
`match`	3.10	`SyntaxError`
`import tomllib`	3.11	`ModuleNotFoundError`
`type X = …`	3.12	`SyntaxError`

> `requires-python = ">=3.14"`: fail at **install time**, with a clear message, not in production.

Environments

One site-packages per interpreter

Project A needs `pandas 1.5`

Project B needs `pandas 2.3`

↘ ↙

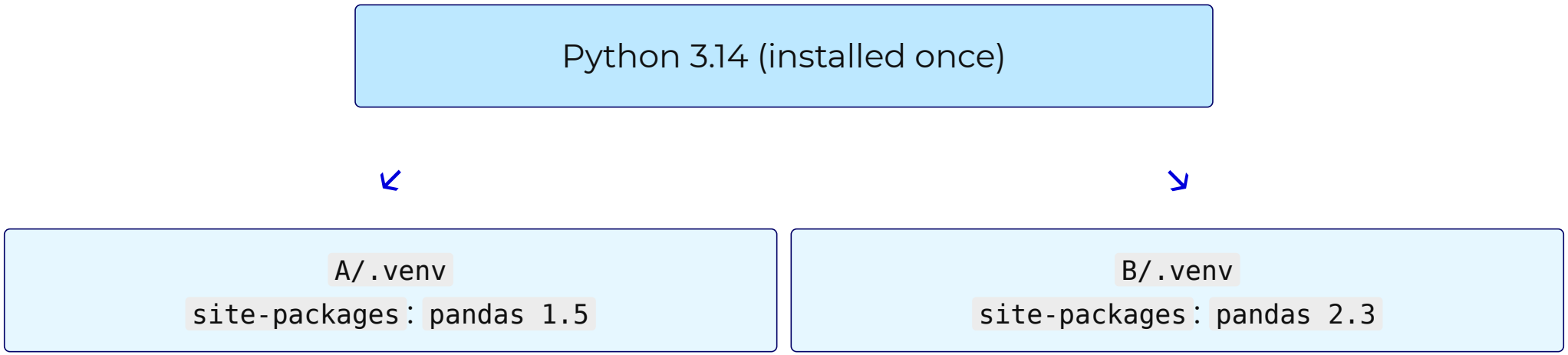
`python3` · `site-packages`: one `pandas`

↓

`pip install` the other one

one of the two projects breaks

Virtual environments



- A `.venv` folder per project: a link to an interpreter + its **own** `site-packages`
- `uv run` uses it directly: nothing to activate, nothing to forget
- **Disposable** and **machine-specific**: rebuilt, never copied or committed

pyproject.toml

```toml
[project]
name = "ledger"
requires-python = ">=3.14"
dependencies = []

[project.scripts]
ledger = "ledger:main"
```

`uv run ledger` → `.venv/bin/ledger` → `main()` in `src/ledger/`

uv: one tool for all of it

`uv python install 3.14`	install a Python version (not the system's)
`uv init --package`	create a project: `pyproject.toml`, `.python-version`, `src/`
`uv run ledger`	run in the project's environment (creates `.venv` if needed)
`uv python pin 3.14`	change the project's Python version

> Replaces pyenv + venv + pip + pipx. Written in Rust: seconds, not minutes.

Lab 3: the ledger becomes a Python project

Today’s lab

Install **uv** and Python 3.14 → `uv init` `ledger` project → `uv run ledger` creates `.venv` → `uv python pin 3.13` refused

`.venv` holds the project’s own Python and packages: rebuilt on each machine, never copied.